

OpenACC - Exercises

NVIDIA HPC-Toolkit OpenACC tutorial

Some more examples can be found on the main documentation of NVIDIA about OpenACC here (<https://docs.nvidia.com/hpc-sdk/compiler/openacc-gs/index.html>).

Guide and Best Practices

Here (<https://openacc-best-practices-guide.readthedocs.io/en/latest/>)

Stupid Makefile for all .c files

```
1 CC=nvc
2
3 CFLAGS= -g -O0 -acc -gpu=cc75 -Minfo=all -acc=noautopar
4
5 # Assume all .c files correspond to a program to build.
6 TARGETS=$(patsubst %.c,%, $(wildcard *.c))
7
8 all: $(TARGETS)
9
10 # This is a static pattern rule to build all .c programs (TARGETS).
11 # Don't define a recipe. Instead, rely on implicit rules to build these. If
12 # we need to give more specific recipes, we can override the implicit rule
13 # by writing a recipe for the target.
14 $(TARGETS): %: %.c
15     $(CC) $(CFLAGS) -o $@.x {{{html}}}lt;
16
17 clean:
18     rm -f $(addsuffix .x, $(TARGETS))
```

(Stupid) Makefile v2

Use for multiple compilers single .c source

```

1  # Leave the following to the beginning of makefile for default target
2  # .PHONY: default
3  # default: nvc
4  .PHONY: default
5  default: nvc
6
7  # Set no optimization -O0 and debug symbols -g
8  CFLAGS = -g -O0
9
10 #In case of problems with the stack for gcc -Wl,--stack,1000000000
11 GCC = gcc
12 FOR = gfortran
13 # -acc=noautopar is needed to avoid automatic parallelization of loops
14 # -gpu=cc75 is for "our" tesla
15 NVCC = nvc $(CFLAGS) -acc -gpu=cc75 -Minfo=all -acc=noautopar
16 NVF90 = nvfortran $(CFLAGS) -acc -gpu=cc75 -Minfo=all -acc=noautopar
17
18 help:
19     @echo "Please specify the make target"
20     @echo "make gcc or make gfor or make nvc or make nvf"
21     @echo "make nvc is the default"
22
23 nvc: *.c
24     $(NVCC) -o $(basename {{{html}}}lt;).x {{{html}}}lt;
25
26 nvf: *.F90
27     $(NVF90) -o $(basename {{{html}}}lt;).x {{{html}}}lt;
28
29 gcc: *.c
30     $(GCC) -o $(basename {{{html}}}lt;).x {{{html}}}lt;
31
32 gfor: *.f90
33     $(FOR) -o $(basename {{{html}}}lt;).x {{{html}}}lt;
34
35 clean:
36     $(RM) *.x
37

```

DAXPY

What does DAXPY means: calculates the product of a scalar and a vector

$$\vec{D} = a\vec{X} + \vec{Y}$$

D = double precision vector (output)

a = double precision scalar value (input)

X, Y = double precision vector (input)

p = plus (summation)

00 - Serial Code

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>      /* clock_t, clock, CLOCKS_PER_SEC */
4  #define getClock() ((double)clock() / CLOCKS_PER_SEC)
5
6  int main ( int argc, char** argv ) {
7
8      //Take time
9      double tstart = getClock();
10
11     size_t n = 200000000;
12
13     printf ( "\n" );
14     printf( "The total memory allocated is %7.3lf GB.\n",
15            3.0*sizeof(double)*n/1024/1024/1024 );
16
17     //NOTE: statically allocated arrays are allocatee in stack, this could be prob
18     //check with: ulimit -s
19     //if stack is low set to unlimited: ulimit -s unlimited
20     double d[n], x[n], y[n];
21     const double a = 16.0;
22
23     printf ( "\n" );
24     printf ( "DAXPY - C code - serial\n" );
25     printf ( "\n" );
26
27     /* .....Allocate the vector data ..... */
28     for (size_t i=0; i<n; i++) {
29         x[i] = 1.0;
30         y[i] = 2.0;
31     }
32
33     /* .....Main cycle ..... */
34
35     for (size_t i = 0; i < n; i++)
36         d[i] = a*x[i] + y[i];
37
38
39     /* .....Print the first 10 results ..... */
40     printf( "\n" );
41     printf( " %2s %10s %10s %10s\n", "i", "X", "Y", "D" );
42     printf( "\n" );
43     for (int i = 0; i < n && i < 10; i++ )
44         printf( " %2d %10.4f %10.4f %10.4f\n", i, x[i], y[i], d[i] );
45     printf ( "\n" );
46
47     //Take time
48     double tstop = getClock();
49
50     printf("Work time (s) = %.6f\n\n", tstop-tstart );
51
52     /* .....Terminate .....*/
53     return 0;

```

```
54 |  
55 }
```

Compiler Info

38, FMA (fused multiply-add) instruction(s) generated

Time

Work time (s) = 1.848768

01 - OpenACC: kernels

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>      /* clock_t, clock, CLOCKS_PER_SEC */
4  #define getClock() ((double)clock() / CLOCKS_PER_SEC)
5
6  int main ( int argc, char** argv ) {
7
8      //Take time
9      double tstart = getClock();
10
11     size_t n = 200000000;
12
13     printf ( "\n" );
14     printf( "The total memory allocated is %7.3lf GB.\n",
15            3.0*sizeof(double)*n/1024/1024/1024 );
16
17     //NOTE: statically allocated arrays are allocated in stack, this could be prob
18     //check with: ulimit -s
19     //if stack is low set to unlimited: ulimit -s unlimited
20     double d[n], x[n], y[n];
21     const double a = 16.0;
22
23     printf ( "\n" );
24     printf ( "DAXPY - C code - kernels\n" );
25     printf ( "\n" );
26
27     /* .....Allocate the vector data ..... */
28     #pragma acc kernels
29     for (size_t i=0; i<n; i++) {
30         x[i] = 1.0;
31         y[i] = 2.0;
32     }
33
34     /* .....Main cycle ..... */
35
36     #pragma acc kernels
37     for (size_t i = 0; i < n; i++)
38         d[i] = a*x[i] + y[i];
39
40
41     /* .....Print the first 10 results ..... */
42     printf( "\n" );
43     printf( "  %2s  %10s  %10s  %10s\n", "i", "X", "Y", "D" );
44     printf( "\n" );
45     for (int i = 0; i < n && i < 10; i++ )
46         printf( "  %2d  %10.4f  %10.4f  %10.4f\n", i, x[i], y[i], d[i] );
47     printf ( "\n" );
48
49     //Take time
50     double tstop = getClock();
51
52     printf("Work time (s) = %.6f\n\n", tstop-tstart );
53

```

```

54      /* .....Terminate .....*/
55      return 0;
56
57  }

```

kernel: The kernels construct identifies a region of code that may contain parallelism, but relies on the automatic parallelization capabilities of the compiler to analyze the region, identify which loops are safe to parallelize, and then accelerate those loops. Developers will little or no parallel programming experience, or those working on functions containing many loop nests that might be parallelized will find the kernels directive a good starting place for OpenACC acceleration.

Compiler Info

```

main:
  25, Generating implicit copyout(y[:200000000],x[:200000000]) [if not already pres
  29, Loop is parallelizable
      Generating NVIDIA GPU code
      29, #pragma acc loop gang, vector(128) /* blockIdx.x threadIdx.x */
  32, Generating implicit copyout(d[:200000000]) [if not already present]
      Generating implicit copyin(y[:200000000],x[:200000000]) [if not already prese
  37, Loop is parallelizable
      Generating NVIDIA GPU code
      37, #pragma acc loop gang, vector(128) /* blockIdx.x threadIdx.x */
  38, FMA (fused multiply-add) instruction(s) generated

```

Time

Work time (s) = 4.032630

Some comments

- **copyout:** Create space for the listed variables on the device but do not initialize them. At the end of the region, copy the results back to the host and release the space on the device.
- **copyin:** Create space for the listed variables on the device, initialize the variable by copying data to the device at the beginning of the region, and release the space on the device when done without copying the data back the the host.
- **What happens if I wrap both cycle with kernels directive under the same scope???**

Compiler Info v2

main:

```
29, Generating implicit copyout(d[:200000000],y[:200000000],x[:200000000]) [if no
31, Loop is parallelizable
    Generating NVIDIA GPU code
    31, #pragma acc loop gang, vector(128) /* blockIdx.x threadIdx.x */
38, Loop is parallelizable
    Generating NVIDIA GPU code
    38, #pragma acc loop gang, vector(128) /* blockIdx.x threadIdx.x */
39, FMA (fused multiply-add) instruction(s) generated
```



Time v2

Work time (s) = 3.372960

02 - OpenACC: parallel

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>      /* clock_t, clock, CLOCKS_PER_SEC */
4  #define getClock() ((double)clock() / CLOCKS_PER_SEC)
5
6  int main ( int argc, char** argv ) {
7
8      //Take time
9      double tstart = getClock();
10
11     size_t n = 200000000;
12
13     printf ( "\n" );
14     printf( "The total memory allocated is %7.3lf GB.\n",
15            3.0*sizeof(double)*n/1024/1024/1024 );
16
17     //NOTE: statically allocated arrays are allocated in stack, this could be prob
18     //check with: ulimit -s
19     //if stack is low set to unlimited: ulimit -s unlimited
20     double d[n], x[n], y[n];
21     const double a = 16.0;
22
23     printf ( "\n" );
24     printf ( "DAXPY - C code - parallel\n" );
25     printf ( "\n" );
26
27     /* .....Allocate the vector data ..... */
28     #pragma acc parallel
29     {
30
31         for (size_t i=0; i<n; i++) {
32             x[i] = 1.0;
33             y[i] = 2.0;
34         }
35
36         /* .....Main cycle ..... */
37
38         for (size_t i = 0; i < n; i++)
39             d[i] = a*x[i] + y[i];
40
41     }
42
43     /* .....Print the first 10 results ..... */
44     printf( "\n" );
45     printf( " %2s %10s %10s %10s\n", "i", "X", "Y", "D" );
46     printf( "\n" );
47     for (int i = 0; i < n && i < 10; i++ )
48         printf( " %2d %10.4f %10.4f %10.4f\n", i, x[i], y[i], d[i] );
49     printf ( "\n" );
50
51     //Take time
52     double tstop = getClock();
53

```

```

54     printf("Work time (s) = %.6f\n\n", tstop-tstart );
55
56     /* .....Terminate .....*/
57     return 0;
58
59 }

```

Compiler Info

main:

```

29, Generating implicit firstprivate(n)
    Generating NVIDIA GPU code
    31, #pragma acc loop seq
    38, #pragma acc loop seq
29, Generating implicit copyout(d[:200000000],y[:200000000],x[:200000000]) [if no
31, Loop is parallelizable
38, Loop is parallelizable
39, FMA (fused multiply-add) instruction(s) generated

```

Some Comments

The **firstprivate** clause is allowed on the *parallel* and *serial* constructs; it declares that a copy of each item on the list will be created for each gang, and that the copy will be initialized with the value of that item on the local thread when a parallel or serial construct is encountered.

loop seq do loop in sequence (do not divide in chunks)

Timing

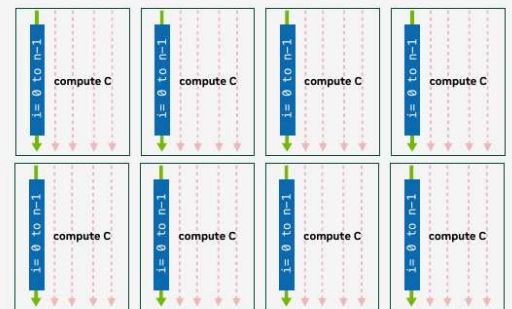
∞

Parallel: a parallel region of code. The compiler generates a parallel kernel for that region. Each gang will execute the entire loop

```
#pragma acc parallel
{
    for (int i=0; i<N; i++)
        c[i] = a[i] + b[i]
}
```

```
#pragma acc parallel
for (int i=0; i<N; i++)
    c[i] = a[i] + b[i]
```

Device



03 - OpenACC: parallel loop

Parallel: a parallel region of code. The compiler generates a parallel kernel for that region. Each gang will execute the entire loop

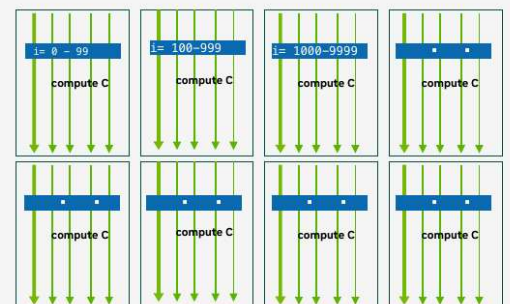
Loop: identifies a loop that should be distributed across threads. Parallel and loop are often placed together

```
#pragma acc parallel
{
    #pragma acc loop
    for (int i=0; i<N; i++)
        c[i] = a[i] + b[i]
}
```

```
#pragma acc parallel
{
    for (int i=0; i<N; i++)
        a[i] = 0;

    #pragma acc loop
    for (int i=0; i<N; i++)
        a[i]++;
}
```

Device



04 - OpenACC: kernels loop

05 - OpenACC restricted

restrict is a keyword, introduced by the C99 standard, that can be used in pointer declarations. By adding this type qualifier, a programmer hints to the compiler that for the lifetime of the pointer, no other pointer will be used to access the object to which it points

Best Practice: C programmers should use the restrict keyword (or the `__restrict` decorator in C++) whenever possible to inform the compiler that the pointers are not aliased, which will frequently give the compiler enough information to then parallelize loops that it would not have otherwise. In addition to the restrict keyword, declaring constant variables using the `const` keyword may allow the compiler to use a read-only memory for that variable if such a

memory exists on the accelerator. Use of `const` and `restrict` is a good programming practice in general, as it gives the compiler additional information that can be used when optimizing the code.

06 - Reduction

OpenACC Reduction construct

Compiler is often very good at detecting when reduction is needed

Operator	Description	Example
<code>+</code>	Addition/Summation	<code>reduction(+:sum)</code>
<code>*</code>	Multiplication/Product	<code>reduction(*:product)</code>
<code>max</code>	Maximum value	<code>reduction(max:maximum)</code>
<code>min</code>	Minimum value	<code>reduction(min:minimum)</code>
<code>&</code>	Bitwise and	<code>reduction(&:val)</code>
<code> </code>	Bitwise or	<code>reduction(:val)</code>
<code>&&</code>	Logical and	<code>reduction(&&:val)</code>
<code> </code>	Logical or	<code>reduction(:val)</code>

07 - Data Movement - 01, 02, 03, 04

- `copy` - Create space for the listed variables on the device, initialize the variable by copying data to the device at the beginning of the region, copy the results back to the host at the end of the region, and finally release the space on the device when done.
- `copyin` - Create space for the listed variables on the device, initialize the variable by copying data to the device at the beginning of the region, and release the space on the device when done without copying the data back to the host.
- `copyout` - Create space for the listed variables on the device but do not initialize them. At the end of the region, copy the results back to the host and release the space on the device.
- `create` - Create space for the listed variables and release it at the end of the region, but do not copy to or from the device.
- `present` - The listed variables are already present on the device, so no further action needs to be taken. This is most frequently used when a data region exists in a higher-level routine.
- `deviceptr` - The listed variables use device memory that has been managed outside of OpenACC, therefore the variables should be used on the device without any address translation. This clause is generally used when OpenACC is mixed with another programming model, as will be discussed in the interoperability chapter.

In the case of the `copy`, `copyin`, `copyout` and `create` clause, their intended functionality will not occur if the variable referenced already exists within device memory. It may be helpful to think of these clauses as having an implicit `present` clause attached to them, where if the variable is found to be present on the device, the other clause will be ignored. An important example of this behavior is that using the `copy` clause when the variable already exists within device memory will not copy any data between the host and device. There is a different directive for copying data between the host and device from within a data region, and will be discussed shortly.